

Boyce–Codd Normal Form (BCNF)

1 Introduction

Definition: BCNF

A relation schema R is in **Boyce–Codd Normal Form (BCNF)** with respect to a set of functional dependencies F if, for every functional dependency $\alpha \rightarrow \beta$ in F^+ , where $\alpha, \beta \subseteq R$, at least one of the following holds:

1. $\alpha \rightarrow \beta$ is **trivial**, i.e., $\beta \subseteq \alpha$.
2. α is a **superkey** of R .

Key Idea

BCNF eliminates *all redundancy* that can be discovered via functional dependencies. However, other forms of redundancy (e.g., multivalued dependencies) may still remain.

2 Example: Schema Not in BCNF

Example

Consider:

inst_dept(ID, name, salary, dept_name, building, budget)

Given:

dept_name $\rightarrow$ building, budget

Here, dept_name is *not* a superkey, so BCNF is violated.

BCNF Decomposition

We decompose inst_dept into:

department(dept_name, building, budget)

instructor(ID, name, salary, dept_name)

Both relations are in BCNF.

3 BCNF Decomposition Process

BCNF Decomposition Algorithm

1. Start with schema R and F .
2. While there exists a violation:
 - (a) Find a non-trivial dependency $\alpha \rightarrow \beta$ in F^+ such that:
 - α is not a superkey.
 - $\beta \not\subseteq \alpha$.
 - (b) Replace R with:
$$R_1 = \alpha \cup \beta$$
$$R_2 = R - (\beta - \alpha)$$
3. Repeat until all schemas are in BCNF.

4 Testing for BCNF

Testing Steps

1. For each non-trivial FD $\alpha \rightarrow \beta$:
 - Compute α^+ .
 - If α^+ contains all attributes of R , α is a superkey (no violation).
 - Otherwise, BCNF is violated.
2. Only checking F may be enough for original schema, but *not* for decomposed relations — you may need F^+ .

5 Worked Examples

Example 1: Simple BCNF Decomposition

Simple Example

Relation:

$$R = (A, B, C), \quad F = \{A \rightarrow B, B \rightarrow C\}$$

Key = $\{A\}$.

Violation: $B \rightarrow C$ (B is not a superkey).

Decomposition:

$$R_1 = (B, C), \quad R_2 = (A, B)$$

Both R_1 and R_2 are in BCNF.

Example 2: Classroom Example

Classroom Example

Relation:

class(course_id, title, dept_name, credits, sec_id, semester, year,
building, room_number, capacity, time_slot_id)

FDs:

course_id $\rightarrow$ title, dept_name, credits

building, room_number $\rightarrow$ capacity

course_id, sec_id, semester, year $\rightarrow$ building, room_number, time_slot_id

Candidate key: {course_id, sec_id, semester, year}.

Decomposition:

course(course_id, title, dept_name, credits)

classroom(building, room_number, capacity)

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

All resulting relations are in BCNF.

Example 3: Department-Project Assignment

Department-Project Assignment

Relation:

`assign(emp_id, emp_name, dept_id, dept_name, proj_id, proj_name)`

FDs:

$\text{emp_id} \rightarrow \text{emp_name}, \text{dept_id}$

$\text{dept_id} \rightarrow \text{dept_name}$

$\text{proj_id} \rightarrow \text{proj_name}$

Candidate key: $\{\text{emp_id}, \text{proj_id}\}$.

Violations:

- $\text{dept_id} \rightarrow \text{dept_name}$ (dept_id is not a superkey).
- $\text{proj_id} \rightarrow \text{proj_name}$ (proj_id is not a superkey).

Decomposition:

`employee(emp_id, emp_name, dept_id)`

`department(dept_id, dept_name)`

`project(proj_id, proj_name)`

`assignment(emp_id, proj_id)`

All resulting relations are in BCNF.

Example 4: Product-Supplier

Product-Supplier Example

Relation:

product_supply(prod_id, prod_name, supp_id, supp_name, supp_city)

FDs:

$\text{prod_id} \rightarrow \text{prod_name}$

$\text{supp_id} \rightarrow \text{supp_name}, \text{supp_city}$

Candidate key: {prod_id, supp_id}.

Violations:

- $\text{prod_id} \rightarrow \text{prod_name}$ (prod_id not superkey).
- $\text{supp_id} \rightarrow \text{supp_name}, \text{supp_city}$ (supp_id not superkey).

Decomposition:

product(prod_id, prod_name)

supplier(supp_id, supp_name, supp_city)

product_supplier(prod_id, supp_id)

All resulting relations are in BCNF.

Example 5: BCNF Decomposition with $\alpha \rightarrow \beta$

BCNF Decomposition Example

Given relation schema:

$r(\alpha, \beta, \gamma)$

and functional dependency:

$\alpha \rightarrow \beta,$

we decompose r into:

$r_1(\alpha, \beta)$ and $r_2(\alpha, \gamma)$.

a. Expected Constraints:

- α is the **primary key** in both r_1 and r_2 .
- α in r_2 acts as a **foreign key** referencing α in r_1 .

b. Example of Inconsistency Without Foreign-Key Enforcement:

- Suppose r_1 has tuples with $\alpha = a_1, a_2$.
- Without foreign key enforcement, r_2 could have a tuple with $\alpha = a_3$ that does not exist in r_1 .
- This leads to an inconsistency because r_2 references a nonexistent α .

c. 3NF Decomposition Constraints:

- Each decomposed relation has a **primary key** consisting of the determinant X from an FD $X \rightarrow A$.
- **Foreign keys** link relations to preserve referential integrity.
- 3NF decomposition preserves dependencies and lossless join but may retain some redundancy.

Example 6: BCNF Decomposition with $\alpha \rightarrow \beta$

BCNF Decomposition Example

Relation schema:

$$R = (A, B, C, D, E)$$

Functional dependencies:

$$F = \{A \rightarrow BC, \quad CD \rightarrow E, \quad B \rightarrow D\}$$

Decomposition:

$$R_1 = (A, B, C), \quad R_2 = (A, D, E)$$

Part 1: Show the decomposition is lossless

Step 1: Find common attributes

$$R_1 \cap R_2 = \{A\}$$

Step 2: Lossless join condition

The decomposition R_1, R_2 is lossless if:

$$(R_1 \cap R_2) \rightarrow R_1 \quad \text{or} \quad (R_1 \cap R_2) \rightarrow R_2$$

i.e., the common attributes functionally determine all attributes of at least one component.

Step 3: Compute closure A^+ under F

$$\begin{aligned} A &\xrightarrow{A \rightarrow BC} \{A, B, C\} \\ &\xrightarrow{B \rightarrow D} \{A, B, C, D\} \\ &\xrightarrow{CD \rightarrow E} \text{Since } C, D \in \{A, B, C, D\}, \Rightarrow \{A, B, C, D, E\} \end{aligned}$$

Thus,

$$A^+ = \{A, B, C, D, E\} = R$$

Conclusion

Since A (the intersection) functionally determines all attributes in R , it certainly determines all attributes in both R_1 and R_2 . Hence, the decomposition is **lossless**.

Decomposition $R = R_1 \cup R_2$ is lossless

Part 2: Find a lossless BCNF decomposition of R

Step 1: Check BCNF violations in R

Check each FD $\alpha \rightarrow \beta$ for α being a superkey in R :

- $A \rightarrow BC$: $A^+ = R$ (shown above), so A is a key. **No violation.**
- $CD \rightarrow E$: Compute $(CD)^+$:

$$CD \xrightarrow{CD \rightarrow E} \{C, D, E\}$$

Does not contain all attributes of R , so CD is **not a key**. **Violation!**

- $B \rightarrow D$: Compute B^+ :

$$B \xrightarrow{B \rightarrow D} \{B, D\}$$

Not all attributes $\rightarrow$ **Violation!**

- $E \rightarrow A$: Compute E^+ :

$$E \xrightarrow{E \rightarrow A} \{E, A\} \xrightarrow{A \rightarrow BC} \{A, B, C, E\} \xrightarrow{B \rightarrow D} \{A, B, C, D, E\} = R$$

E is a key $\rightarrow$ **No violation.**

—

Step 2: Decompose on violation $CD \rightarrow E$

Decompose R into:

$$R_3 = (C, D, E), \quad R_4 = (A, B, C, D)$$

- In R_3 , $CD \rightarrow E$, and CD is a candidate key for R_3 . **BCNF holds.**

—

Step 3: Check R_4 for BCNF

Functional dependencies projected on R_4 :

$$F_{R_4} = \{A \rightarrow BC, \quad B \rightarrow D\}$$

Check keys for R_4 :

- Try AB :

$$(AB)^+ = \{A, B, C, D\} = R_4$$

So candidate key is AB .

—

Check violations:

- $A \rightarrow BC$: A alone is not a superkey $\rightarrow$ **violation**. - $B \rightarrow D$: B alone is not a superkey $\rightarrow$ **violation**.

—

Step 4: Decompose R_4 on $A \rightarrow BC$

$$R_5 = (A, B, C), \quad R_6 = (A, B, D)$$

- R_5 : $A \rightarrow BC$, A is a key $\rightarrow$ **BCNF holds**.

—

Step 5: Check R_6 for BCNF

FD on R_6 :

$$B \rightarrow D$$

Candidate keys in R_6 :

- AB :

$$(AB)^+ = \{A, B, D\} = R_6$$

Violation because $B \rightarrow D$ and B is not a key.

—

Step 6: Decompose R_6 on $B \rightarrow D$

$$R_7 = (B, D), \quad R_8 = (A, B)$$

- R_7 : $B \rightarrow D$, B is key $\rightarrow$ **BCNF holds**. - R_8 : No non-trivial FDs $\rightarrow$ **BCNF holds**.

—

Final BCNF Decomposition

$$\left\{ \begin{array}{l} R_3 = (C, D, E), \quad CD \rightarrow E \\ R_5 = (A, B, C), \quad A \rightarrow BC \\ R_7 = (B, D), \quad B \rightarrow D \\ R_8 = (A, B) \end{array} \right.$$

Each relation is in BCNF, and the decomposition is lossless.

6 Dependency Preservation in BCNF

Important Note

It is *not always possible* to have a BCNF decomposition that is dependency preserving.

Example:

$$R = (J, L, K), \quad F = \{JK \rightarrow L, L \rightarrow K\}$$

Candidate keys: JK, JL .

No BCNF decomposition can preserve all dependencies without joins.

7 More Examples: Design Goal

This section explores various functional dependency (FD) scenarios for a relation $R = (A, B, C)$, examining the **candidate key**, **decomposition**, and whether the decomposition satisfies:

- Lossless-Join property
- BCNF
- Dependency Preservation

Legend

- ✓ : Property satisfied
- NA : Not applicable (no decomposition needed)
- “No” : Property not satisfied

Sl	FDs	Key	Decomposition	Lossless?	BCNF?	Dependency Preserving?
1	$A \rightarrow B, A \rightarrow C$	A	No decomposition needed	NA	Already	NA
2	$A \rightarrow B, B \rightarrow C$	A	$R_1 = (B, C), R_2 = (A, B)$	✓	✓	✓
3	$A \rightarrow B, C \rightarrow A$	C	$R_1 = (A, B), R_2 = (A, C)$	✓	✓	✓
4	$A \rightarrow B, C \rightarrow B$	AC	$R_1 = (A, B), R_2 = (A, C)$	✓	✓	No (misses $C \rightarrow B$)
			$R_1 = (B, C), R_2 = (A, C)$	✓	✓	No (misses $A \rightarrow B$)
5	$A \rightarrow C, B \rightarrow A$	B	$R_1 = (A, C), R_2 = (A, B)$	✓	✓	✓
6	$A \rightarrow C, C \rightarrow B$	A	$R_1 = (B, C), R_2 = (A, C)$	✓	✓	✓
7	$A \rightarrow C, B \rightarrow C$	AB	$R_1 = (A, C), R_2 = (A, B)$	✓	✓	No
			$R_1 = (B, C), R_2 = (A, B)$	✓	✓	No
8	$B \rightarrow A, B \rightarrow C$	B	No decomposition needed	NA	Already	NA
9	$B \rightarrow A, C \rightarrow A$	BC	$R_1 = (A, B), R_2 = (B, C)$	✓	✓	No
			$R_1 = (A, C), R_2 = (B, C)$	✓	✓	No
10	$B \rightarrow A, C \rightarrow B$	C	$R_1 = (A, B), R_2 = (B, C)$	✓	✓	✓
11	$B \rightarrow C, C \rightarrow A$	B	$R_1 = (A, C), R_2 = (B, C)$	✓	✓	✓
12	$C \rightarrow A, C \rightarrow B$	C	No decomposition needed	NA	Already	NA

Key Observations

- If one attribute functionally determines all others, no decomposition is needed (Cases 1, 8, 12).
- Some decompositions can achieve BCNF and lossless-join but fail dependency preservation (e.g., Cases 4, 7, 9).

- When multiple decompositions are possible, check both for dependency preservation (Cases 4, 7, 9).
- BCNF does not guarantee dependency preservation — a common exam trick.

Exam Tip

In questions where BCNF decomposition is required, always check:

1. **Lossless-join** — ensure $(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$ holds.
2. **Dependency preservation** — verify that all original FDs can be enforced without join.